

1. Introduction

In this project, the primary goal was to detect faces in images and train a classifier to recognize who the faces belonged to. Throughout the project, multiple methods were tested and refined to improve face-detection accuracy and recognition performance. The final pipeline uses YuNet for detection and FaceNet for the embeddings and lastly CatBoost for recognition.

2. Approaches Tested

2.1 First Approach: Dlib Detection & Embeddings + SVM

1. Face Detection and Embeddings:

- **Method:** Used Dlib for both face detection and generating facial embeddings per name.
- **Outcome:**
 - Detection missed many faces or failed to locate faces correctly in certain images.
 - An SVM was trained on the Dlib-based embeddings, but the results were poor, leading to an overall low-accuracy pipeline.

2. Reason for Moving On:

- Many faces were not being detected at all.
 - Even when faces were detected, the SVM on Dlib embeddings performed poorly.
 - Decided to start over with a different detection approach.
-

2.2 Second Approach: Evaluating Multiple Face-Detection Models

1. Testing Detection Models:

- Explored various face detectors (e.g., RetinaFace, Dlib, YuNet, etc.).
- **Conclusion:** YuNet performed best on this dataset (highest detection rate).

2. Image Alignment Attempt:

- Tried aligning faces so they would be perfectly frontal.
- **Outcome:** Found it unnecessary since training the model on various orientations can be beneficial.

3. Dlib Embeddings + Dlib Classifier:

- Attempted again to generate embeddings with Dlib for each name and then train a Dlib-based model on those embeddings.
 - **Outcome:** Results were still lacking, prompting another restart.
-

2.3 Third Approach: YuNet + Keras FaceNet Embeddings + CatBoost

1. Detection with YuNet:

- Confirmed YuNet is reliable for this dataset.
- Adjusted preprocessing steps (e.g., scaling, rotating images when needed) to boost detection quality.

2. Embedding with Keras FaceNet:

- Switched from Dlib to **Keras FaceNet** to generate embeddings.
- Saw much better consistency and higher-quality embeddings.

3. Classifier Comparison & Final Model Choice:

- Tested multiple classifiers, including:
 - SVC (linear),
 - RandomForest,
 - XGBoost,
 - LightGBM,
 - **CatBoost (chosen),**
 - K-Nearest Neighbors,
 - Logistic Regression.
- Settled on **CatBoost** due to strong performance and straightforward tuning.

4. Handling Different Orientations in Test Set:

- Implemented multi-orientation detection so that images turned upside-down or sideways could still be handled properly.

5. Results & Augmentation Experiments:

- Achieved significantly improved face recognition results—detected most faces correctly even on challenging images.
- Tried augmenting the training set with ImageDataGenerator, but it did not yield better performance.
- Errors generally came from:
 - Faces too far away or partially hidden
 - Bad quality images

6. Conclusion & Minor Improvements:

- The pipeline was deemed satisfactory after addressing rotation issues and refining preprocessing.

- Minor tweaks (e.g., adjusting hyperparameters) were made, but the setup was already producing strong results.
-

3. Reasoning for Using (or Not Using) Each Method

1. Dlib Detection & Embeddings:

- Pros: Easy to implement, widely used library.
- Cons: Missed many faces, poor embeddings for this dataset → low SVM accuracy.

2. Multiple Detectors (RetinaFace, Dlib, YuNet...):

- Pros: Comprehensive evaluation; discovered YuNet's strengths.
- Cons: Time-consuming to test all.
- Decision: YuNet gave the best detection results, so it was chosen.

3. Face Alignment:

- Pros: Sometimes helps frontalize faces for recognition.
- Cons: In practice, adds complexity, and our chosen model trained well on slightly varied angles.
- Decision: Skipped alignment for the final pipeline.

4. Keras FaceNet Embeddings:

- Pros: Provided robust features, integratable with various classifiers.
- Cons: Requires more time when training, but it is worth it.

5. CatBoost Classifier:

- Pros: Performed best among tested models, straightforward tuning, handles categorical data well.
- Cons: More complicated than basic SVM or KNN.
- Decision: Final choice due to superior accuracy.

6. Data Augmentation:

- Pros: Can expand small datasets, potentially improving generalization.
 - Cons: In our case, did not help—likely because rotations/flips were already handled during detection or the original dataset was sufficient.
-

4. Final Outcome & Lessons Learned

- **Outcome:**
 - A robust pipeline combining **YuNet for face detection** and **Keras FaceNet + CatBoost** for recognition.
 - Multi-orientation detection handled tilted/rotated images well.
 - FaceNet embeddings was tested to be way better than Dlib-based embeddings on this dataset.
 - Accuracy was satisfactory, with most errors stemming from genuinely unrecognizable or distant faces.
- **Lessons Learned:**
 - Always test multiple face-detection methods, as performance can vary drastically across datasets.
 - Aligning faces is not always necessary; more diverse training data can improve model robustness.
 - Data augmentation does not guarantee improvements, especially if the pipeline already handles variations in orientation.
 - Embedding quality is crucial for better classification; switching to FaceNet was a significant upgrade.